

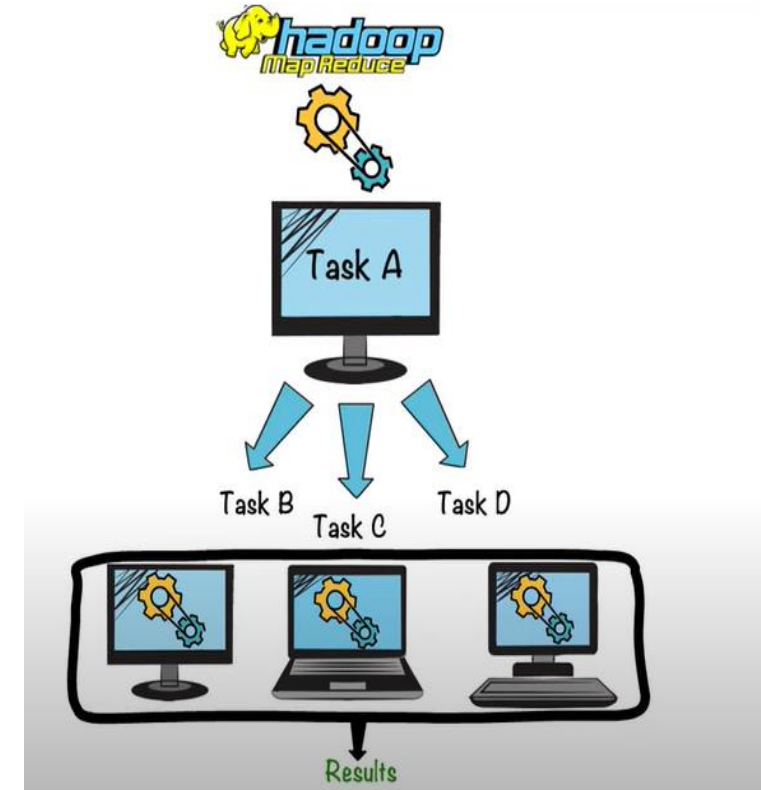
Big Data Frameworks

Batch Processing

Marvin Moughabghab

What is Hadoop MapReduce

Hadoop MapReduce is a software framework for easily writing applications which process vast amounts of data (multi-terabyte data-sets) in-parallel on large clusters (thousands of nodes) of commodity hardware in a reliable, fault-tolerant manner.



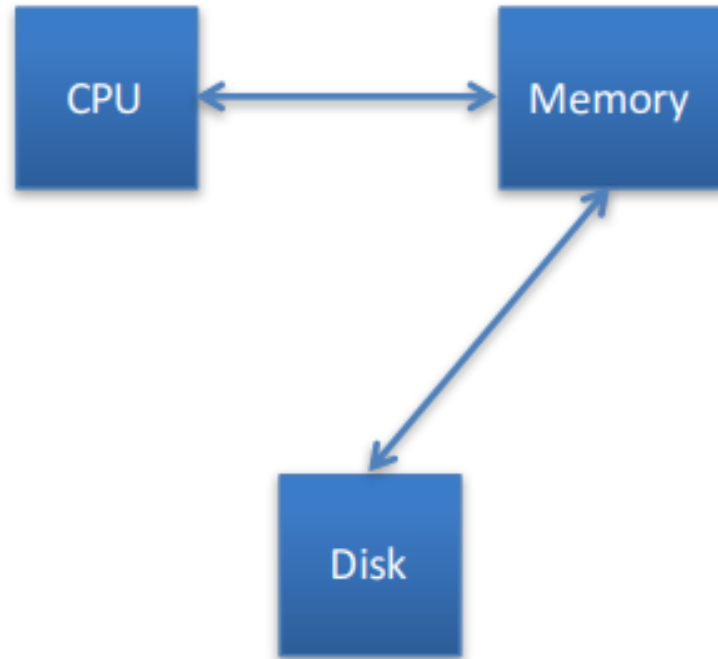


What is Hadoop MapReduce

- MapReduce allows us to:
 - Store data redundantly
 - For persistence and availability
 - Move computation close to data
 - Reduce data movement
 - Simple Programming Model
 - Hides the complexity

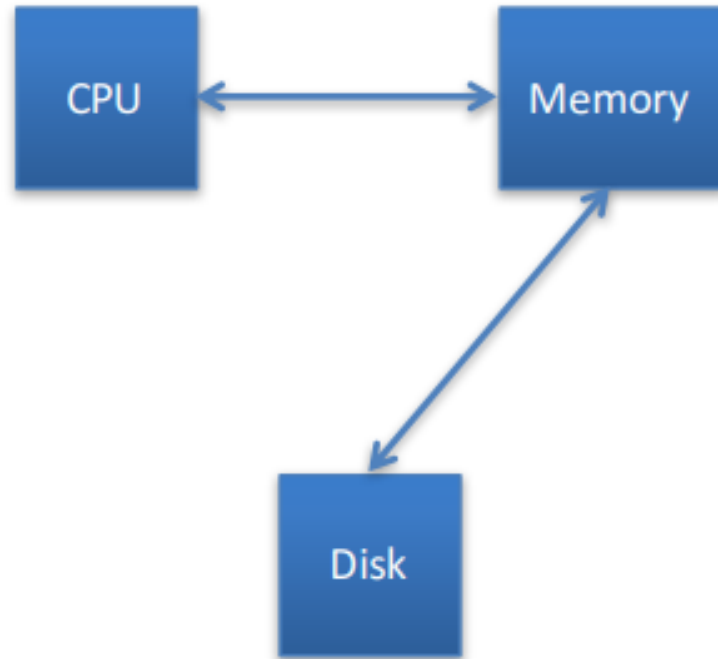


What is Hadoop MapReduce



- Classical computation:
 - Data stored on disk
 - Program runs in CPU
 - Data read from disk into memory
 - Program accesses data in memory

What is Hadoop MapReduce



What happens if data is massive and can not fit in memory?

Swapping

- Data is split into chunks
- Each chunk is loaded into memory and processed

MapReduce computation Model

- Case Study
 - We have a huge text document (TB)
 - Count the number of times each word appears in that file
- Case 1
 - File is too large to fit in memory but pairs can fit
 - Use a HashTable <String, Int>
 - Go through the file and update the HashTable
 - Case 2. pairs don't fit in memory
 - Extract Words → Sort → Count
 - The essence of MapReduce



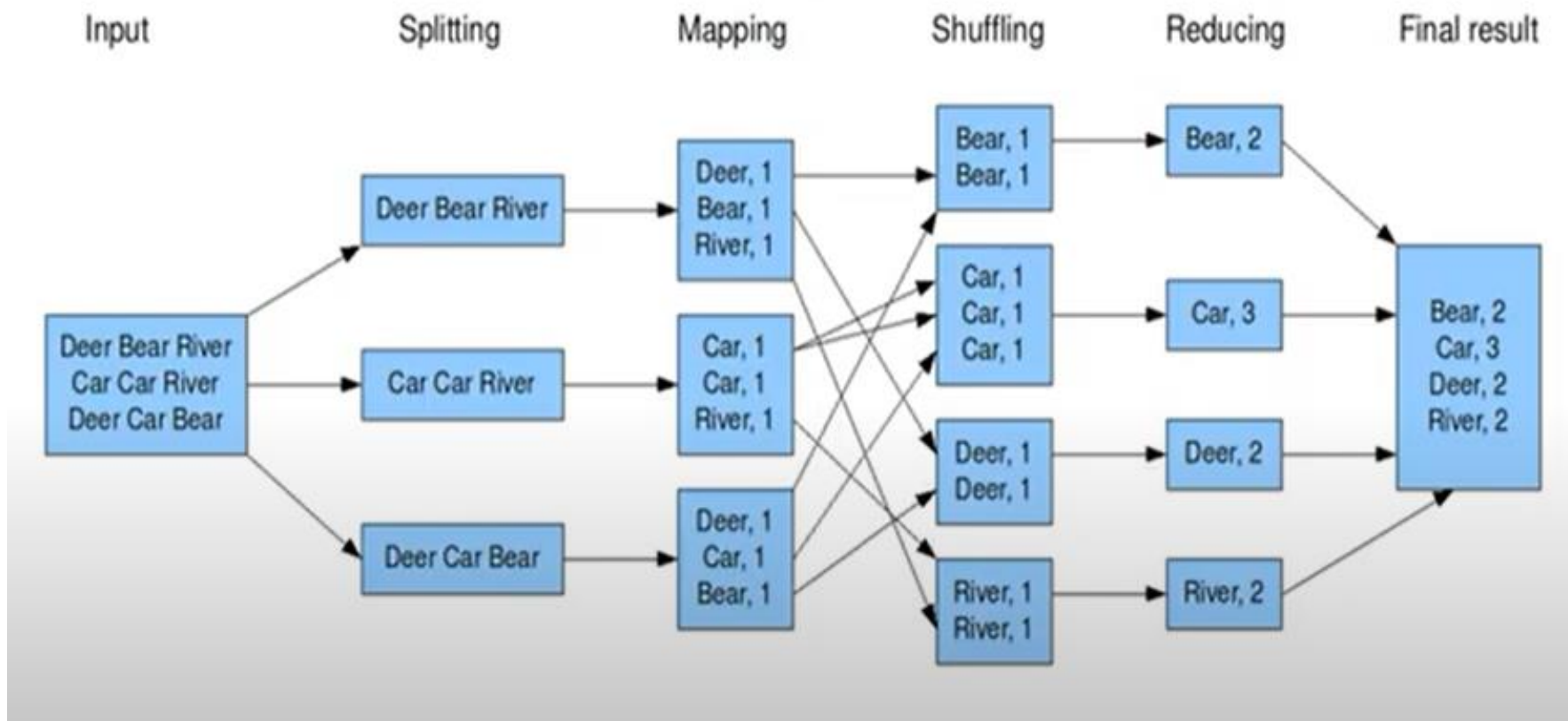
MapReduce computation Model

- Map
 - Scan input file one record at a time
 - Extract required data (keys)
 - Obtain our key-value pairs
- Group by key
 - Sort and Shuffle
- Reduce
 - Aggregate function: Sum, filter, transform
 - Write result



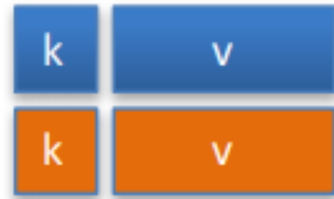
MapReduce computation Model

The overall MapReduce word count process



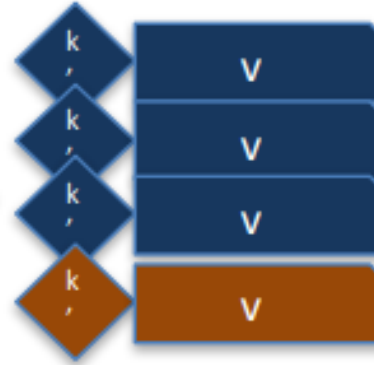
MapReduce – Map Operation

Input key-value pairs

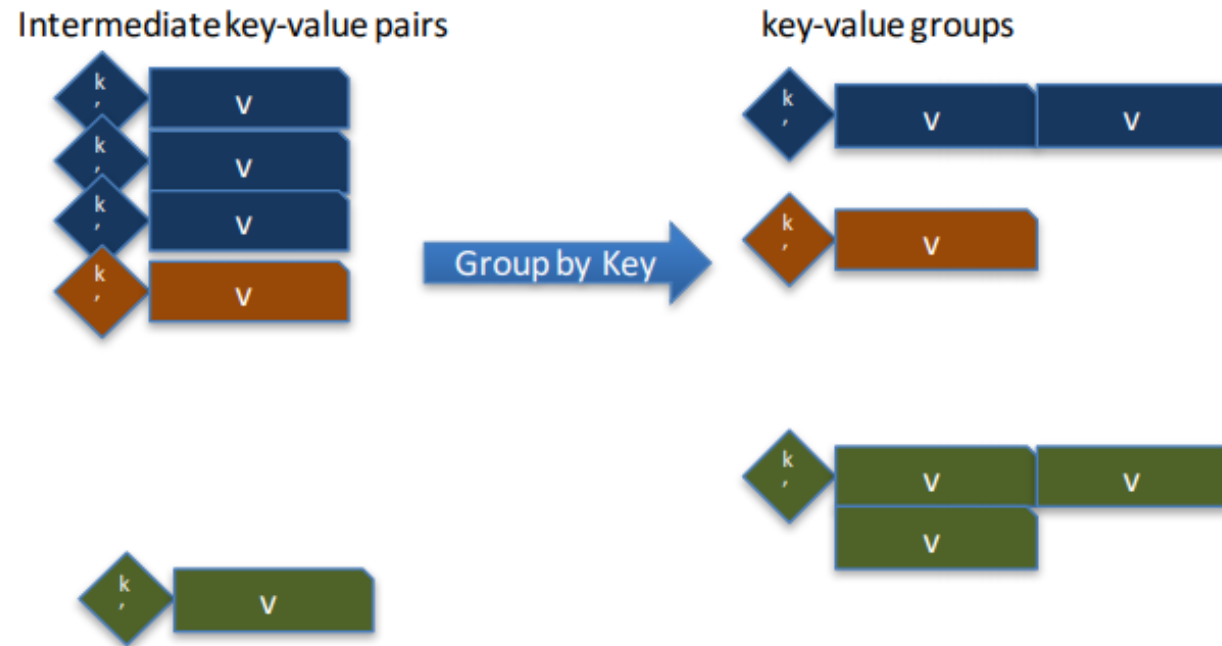


Map

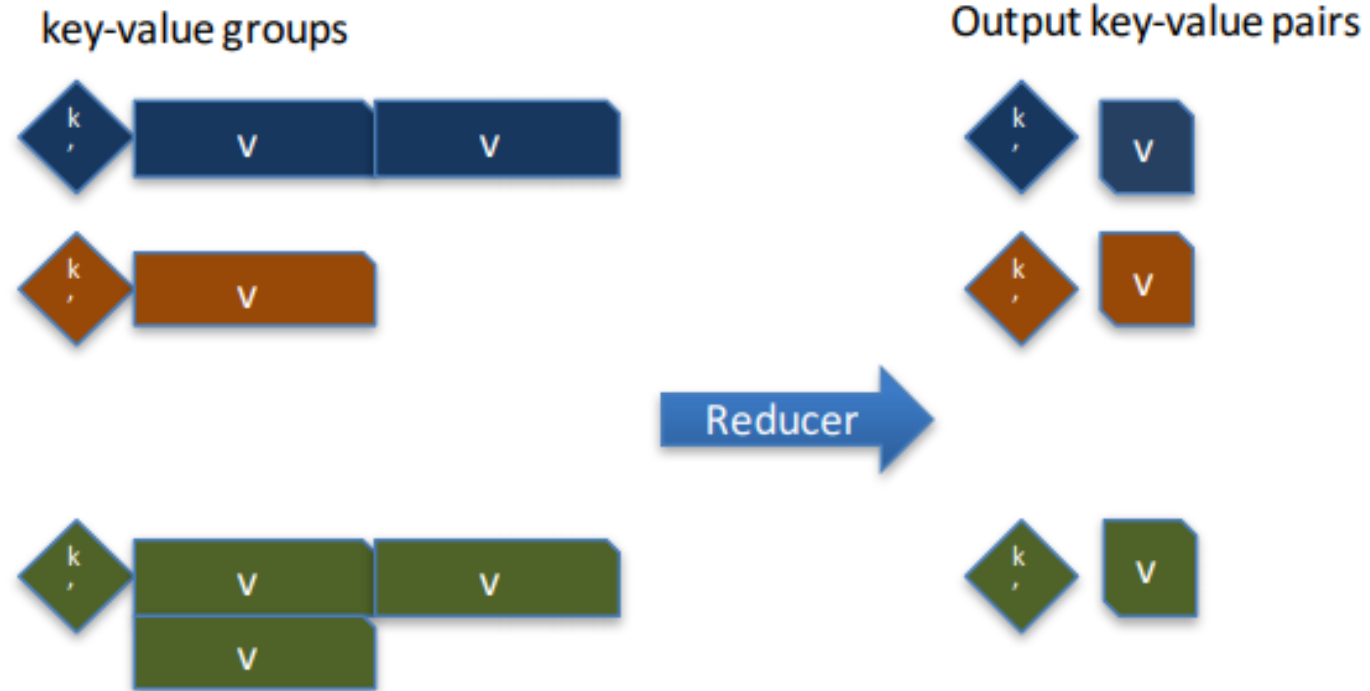
Intermediate key-value pairs



MapReduce – Shuffle Operation



MapReduce – Shuffle Operation





MapReduce – Process

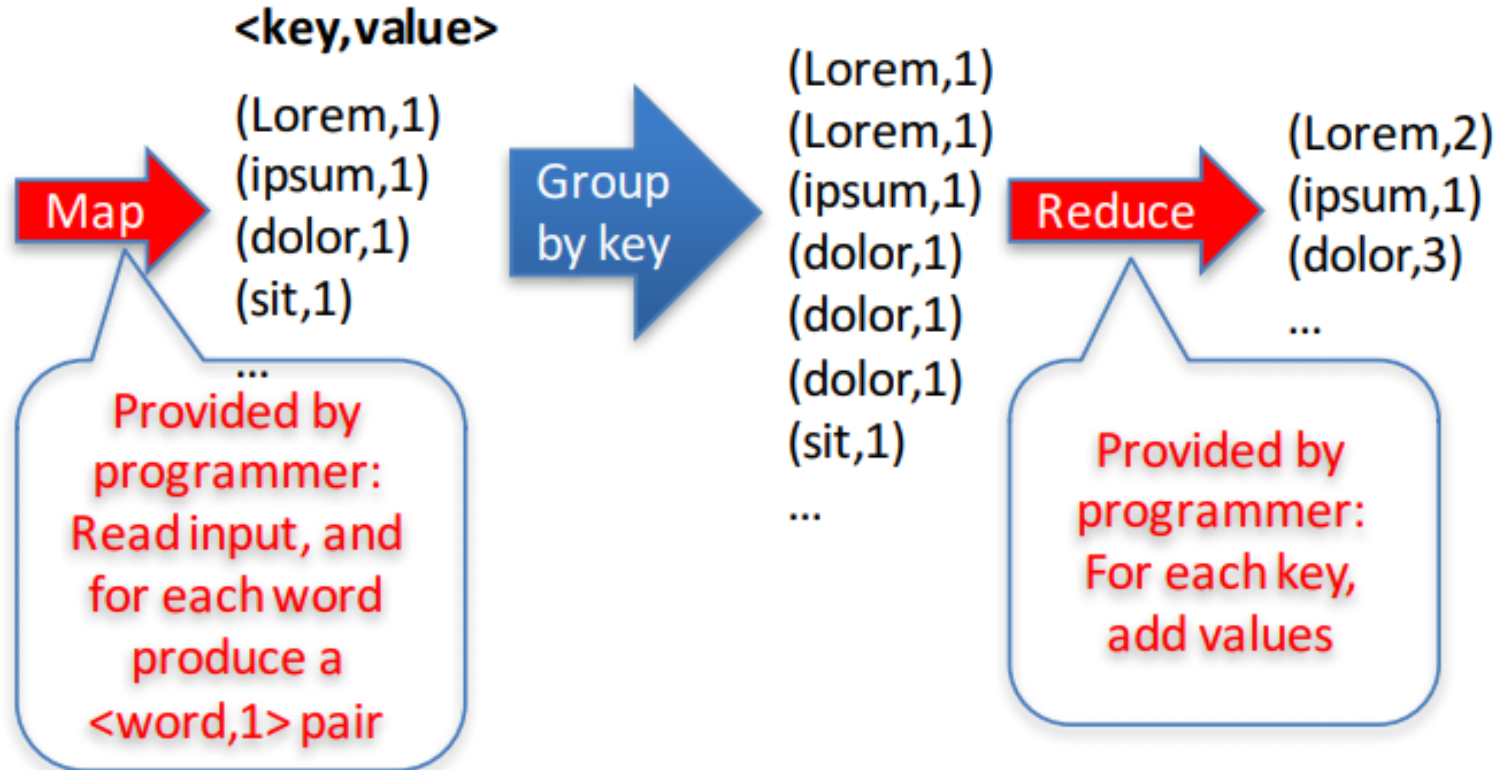
- Input → <key,value> pairs (e.g. Key = Document name, Value= Document Content)
- Programmer specifies two methods:
 - Map(key,value) → <key',value'>*
 - The map function takes a key-value pair and returns an **intermediate** set of key value pair
 - The map call is run on every <key, value> pair
 - Reduce(key',value' *) => <key',value''>*
 - All values value' of the same key key' are reduced together
 - Merges values together to form a smaller set of values
 - There is one reduce call that is ran for each **key'**
 - Typically produces one output value



MapReduce – Word Count example

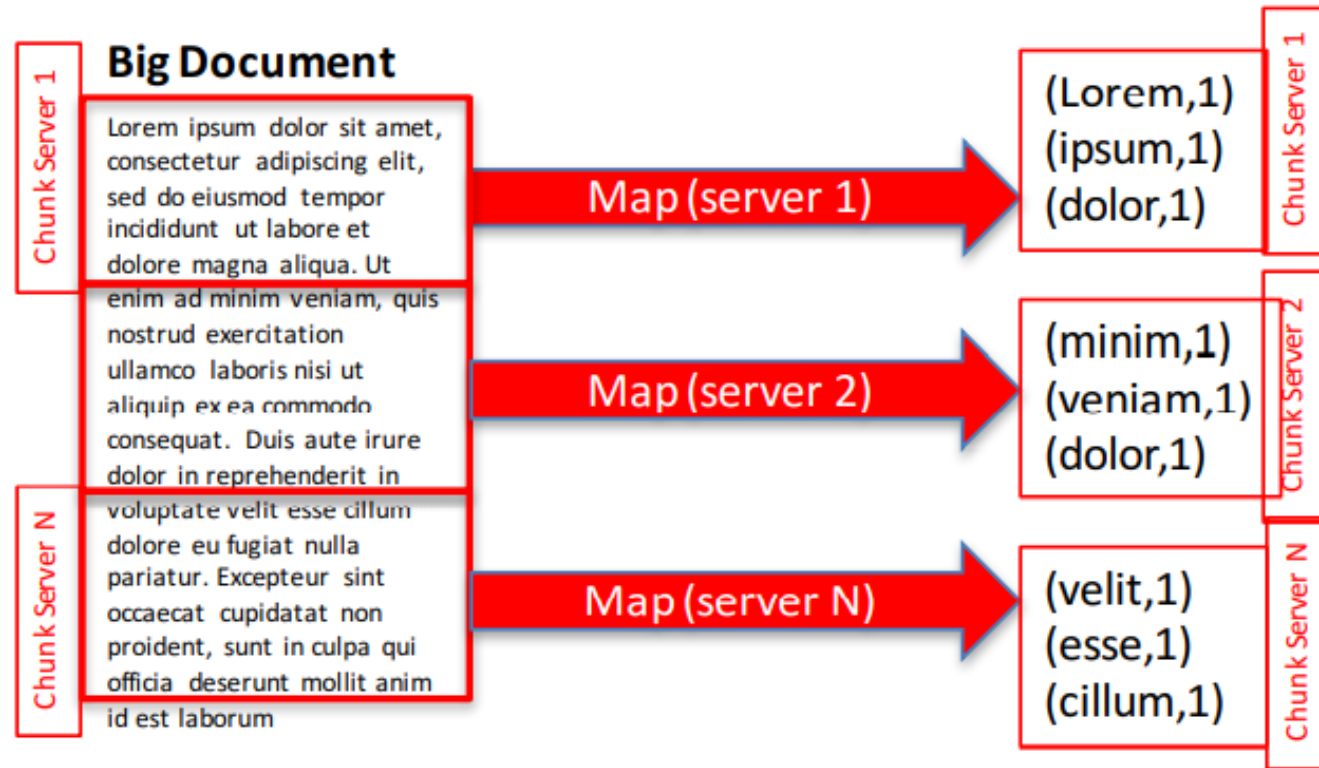
Big Document

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum



MapReduce – Word Count example

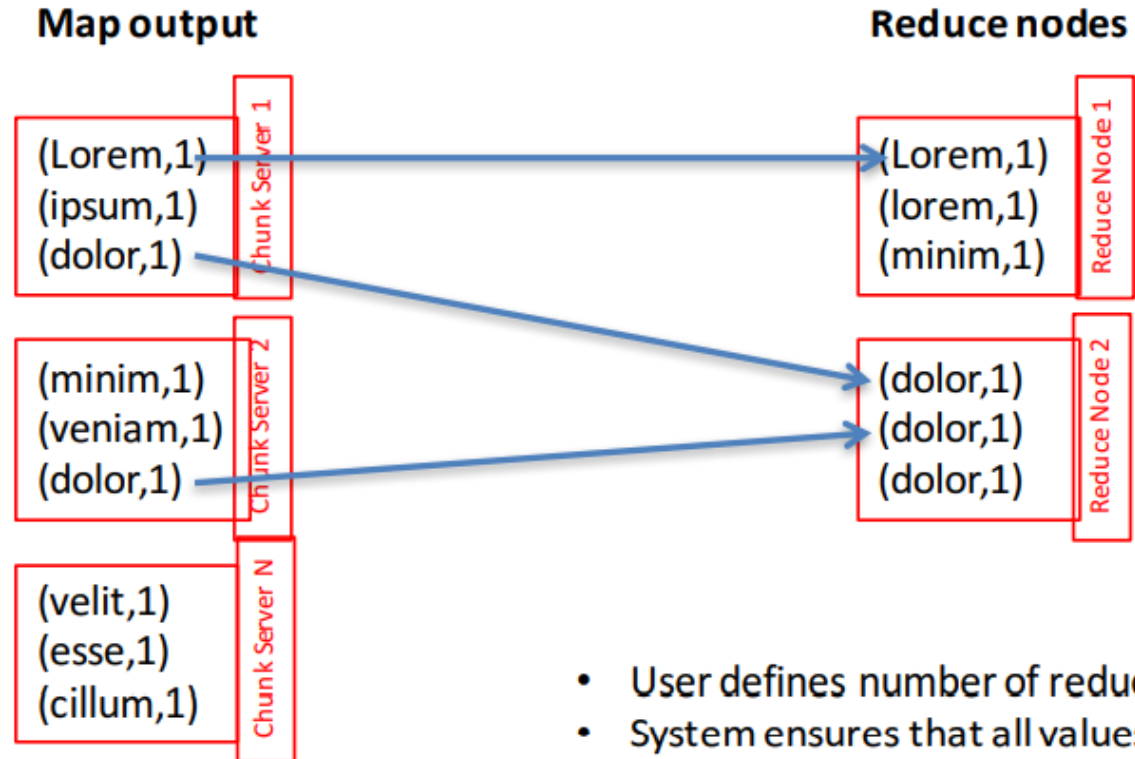
Map Distribution across the nodes



<key', value'>

MapReduce – Word Count example

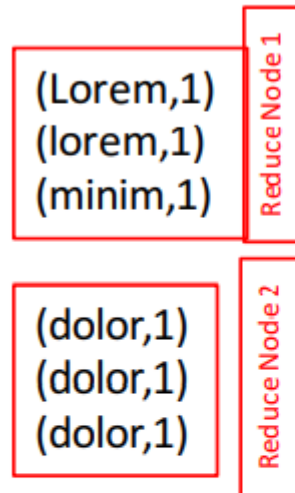
Reduce Distribution across the nodes



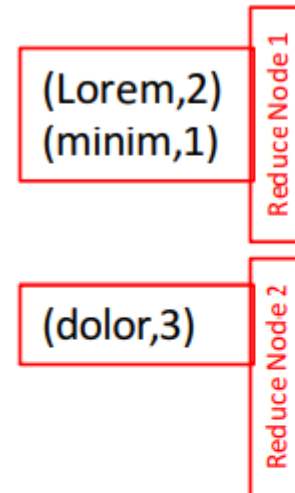
- User defines number of reduce nodes
- System ensures that all values for a given key end up in the same reduce node
 - Done using a Hash function

MapReduce – Word Count example

Reduce nodes



Reduce nodes



- Final result is split among the Reduce Nodes
- OK, since we are using a distributed file system
 - You can still access data as a single file, the Distributed File System handles the complexity



MapReduce – Handling Node Failure

1. Map worker failure
 - Map tasks completed or in-progress at worker are reset to **Idle** (Results are stored on local file system)
 - Idle tasks are rescheduled on other workers
2. Reduce worker failure
 - Only in-progress tasks are reset to idle (no need to do it for completed tasks)
 - Idle tasks restarted on other workers
3. Master Node failure
 - MapReduce tasks is aborted, and client is notified



MapReduce – Summary

MapReduce is responsible of:

Partitioning the input data

Scheduling the execution across a set of machines

Managing inter-machine communication

Performing group and shuffle operations

Handling Node Failures



MapReduce – Number of Jobs

- With M number of map tasks and R the number of reduce tasks
- **Rule of thumb:**
 - M should much larger than the number of nodes in the cluster
 - Reduce task size and speeds up recovery from worker failure
 - Improve load balancing (smaller tasks provide more flexibility in spreading equally on different nodes)
 - Usually R is smaller than M (because output is spread across R files)

MapReduce – Java code example

```
public static class TokenizerMapper extends Mapper<Object, Text, Text,  
IntWritable>{
```

```
    private final static IntWritable one = new IntWritable(1);  
    private Text word = new Text();
```

```
    public void map(Object key, Text value, Context context) throws  
IOException, InterruptedException {
```

```
        StringTokenizer itr = new StringTokenizer(value.toString());  
        while (itr.hasMoreTokens()) {  
            word.set(itr.nextToken());  
            context.write(word, one);  
        }  
    }  
}
```

MapReduce – Java code example

```
public static class IntSumReducer extends  
    Reducer<Text,IntWritable,Text,IntWritable> {  
    private IntWritable result = new IntWritable();  
  
    public void reduce(Text key, Iterable<IntWritable> values, Context context)  
        throws IOException, InterruptedException {  
        int sum = 0;  
        for (IntWritable val : values) {  
            sum += val.get();  
        }  
        result.set(sum);  
        context.write(key, result);  
    }  
}
```



MapReduce – Java code example

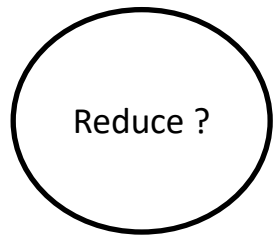
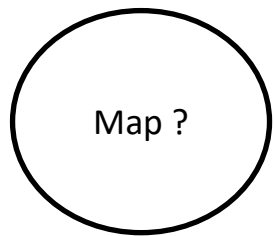
```
Configuration conf = new Configuration();  
Job job = Job.getInstance(conf, "word count");  
job.setJarByClass(WordCount.class);  
job.setMapperClass(TokenizerMapper.class);  
job.setCombinerClass(IntSumReducer.class);  
job.setReducerClass(IntSumReducer.class);  
job.setOutputKeyClass(Text.class);  
job.setOutputValueClass(IntWritable.class);  
FileInputFormat.addInputPath(job, new Path(args[0]));  
FileOutputFormat.setOutputPath(job, new Path(args[1]));  
System.exit(job.waitForCompletion(true) ? 0 : 1);
```



MapReduce – Use Case Example I

Customer Orders

Design the Map and Reduce function in order to study the consumption of each customer



```
{ _id: 1, cust_id: "Ant O. Knee", ord_date: new Date("2020-03-01"), price: 25, items: [ { sku: "oranges", qty: 5, price: 2.5 }, { sku: "apples", qty: 5, price: 2.5 } ], status: "A" },
```

```
{ _id: 2, cust_id: "Ant O. Knee", ord_date: new Date("2020-03-08"), price: 70, items: [ { sku: "oranges", qty: 8, price: 2.5 }, { sku: "chocolates", qty: 5, price: 10 } ], status: "A" },
```

```
{ _id: 3, cust_id: "Busby Bee", ord_date: new Date("2020-03-08"), price: 50, items: [ { sku: "oranges", qty: 10, price: 2.5 }, { sku: "pears", qty: 10, price: 2.5 } ], status: "A" },
```

```
{ _id: 4, cust_id: "Busby Bee", ord_date: new Date("2020-03-18"), price: 25, items: [ { sku: "oranges", qty: 10, price: 2.5 } ], status: "A" },
```

```
{ _id: 5, cust_id: "Busby Bee", ord_date: new Date("2020-03-19"), price: 50, items: [ { sku: "chocolates", qty: 5, price: 10 } ], status: "A"},
```

```
{ _id: 6, cust_id: "Cam Elot", ord_date: new Date("2020-03-19"), price: 35, items: [ { sku: "carrots", qty: 10, price: 1.0 }, { sku: "apples", qty: 10, price: 2.5 } ], status: "A" },
```

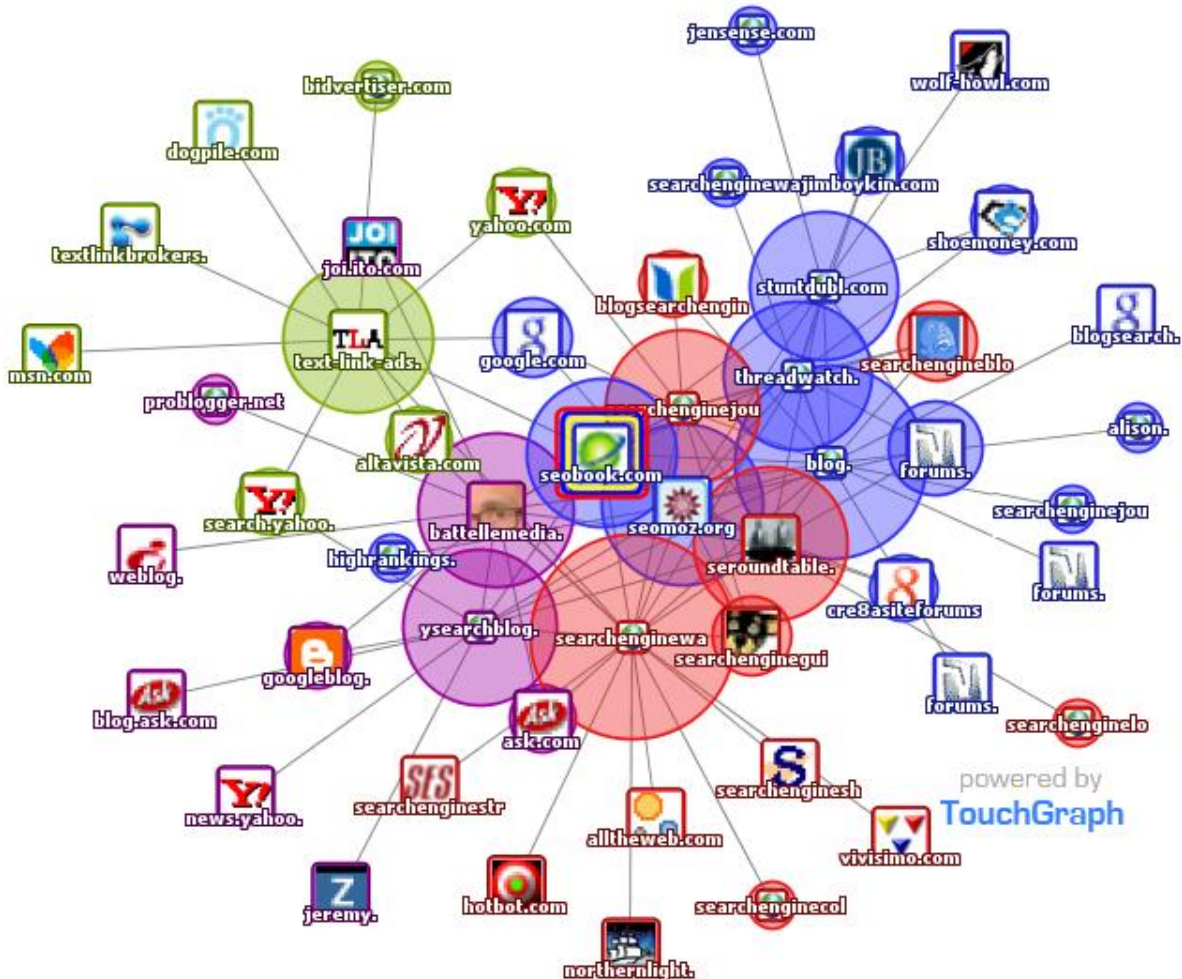
MapReduce – Use Case Example II

Reverse Web Link Graph

Design the Map and Reduce function in order to study the relevancy of a list of big emerging websites by building a Web link graph

Map ?

Reduce ?





MapReduce – Refinement: Combiners

1. Combiners

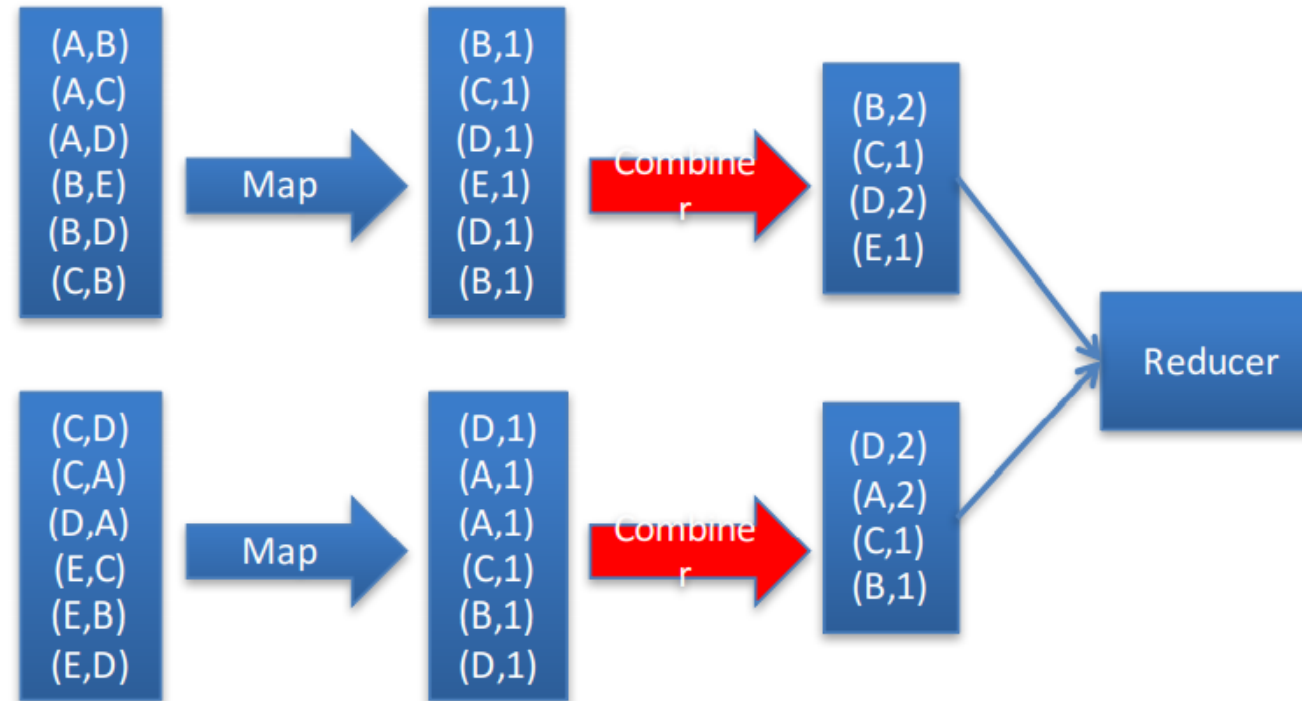
Why ?

- Often a Map task will produce many pairs for the same key k
 - Popular words like “the”
 - Will have to move ALL the tuples to reduce node (exhaustive)
 - Why not reduce data movement ? (Let's have our data pre-aggregated !)

What ?

- Can save network time by pre-aggregating values in the mapper
- Combine ($k, \text{list}(v1)$) $\rightarrow v2$
- Combiner is usually same as reduce function

MapReduce – Refinement : Combiners



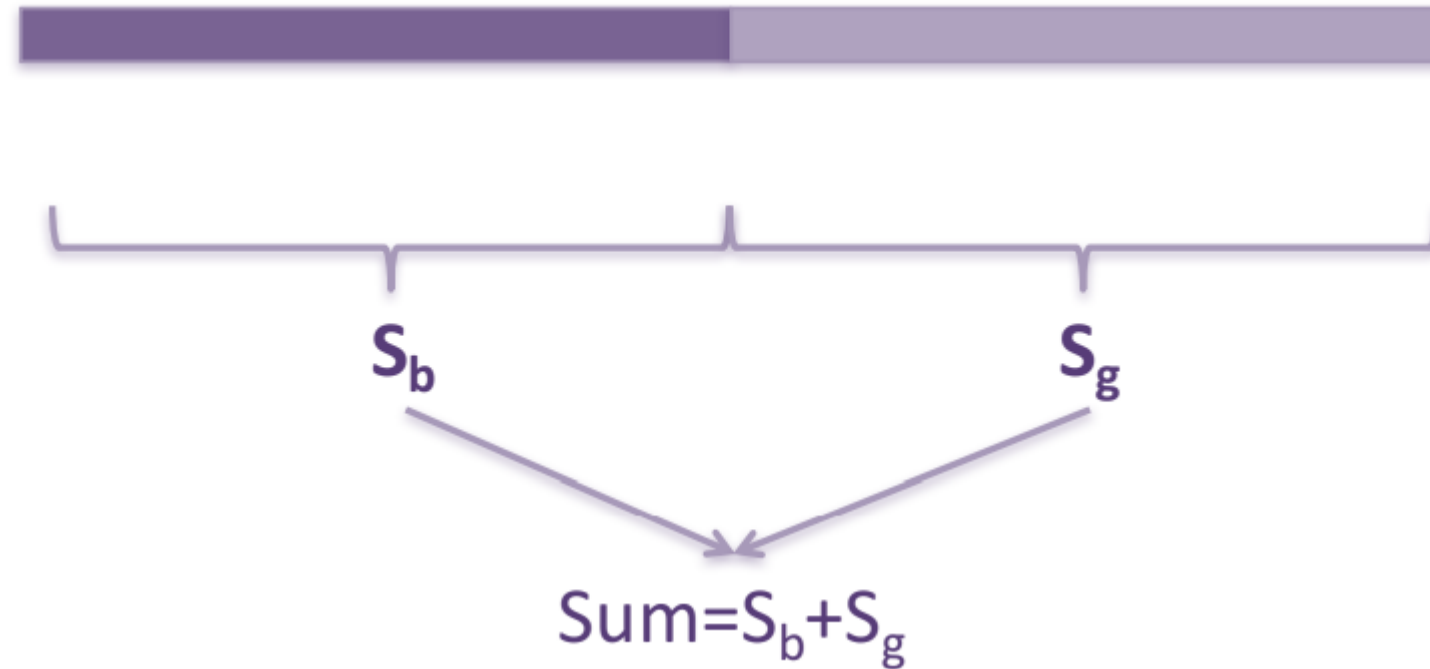


MapReduce – Refinement : Combiners

- Combiner combines the value of all keys of a single mapper on a single node
- Much less data will be copied and shuffled
- Combiner trick works only if reduce function is commutative and associative
 - Sum: $a+b = b+a$ $a+(b+c) = (a+b)+c$
 - Average X

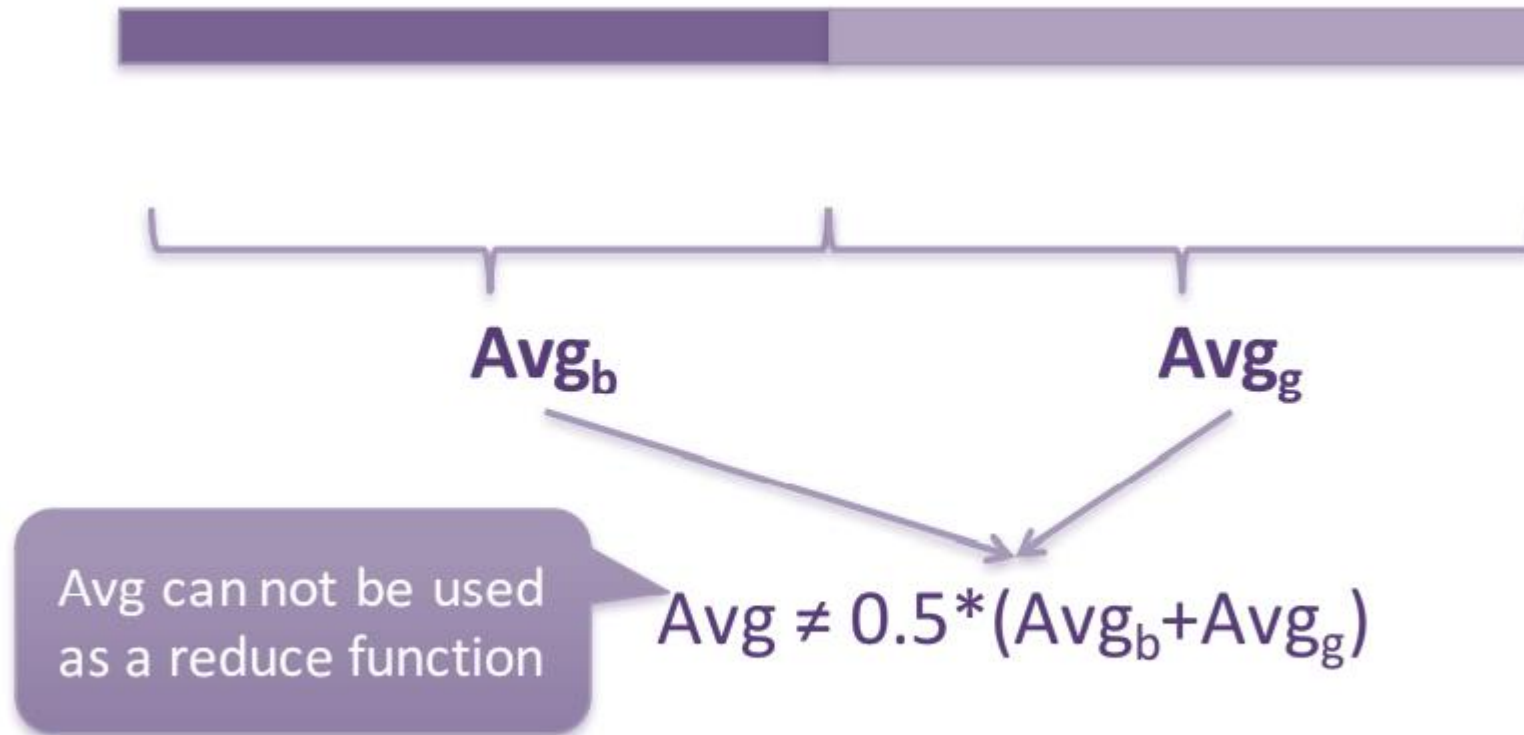
MapReduce – Refinement : Combiners

Sum



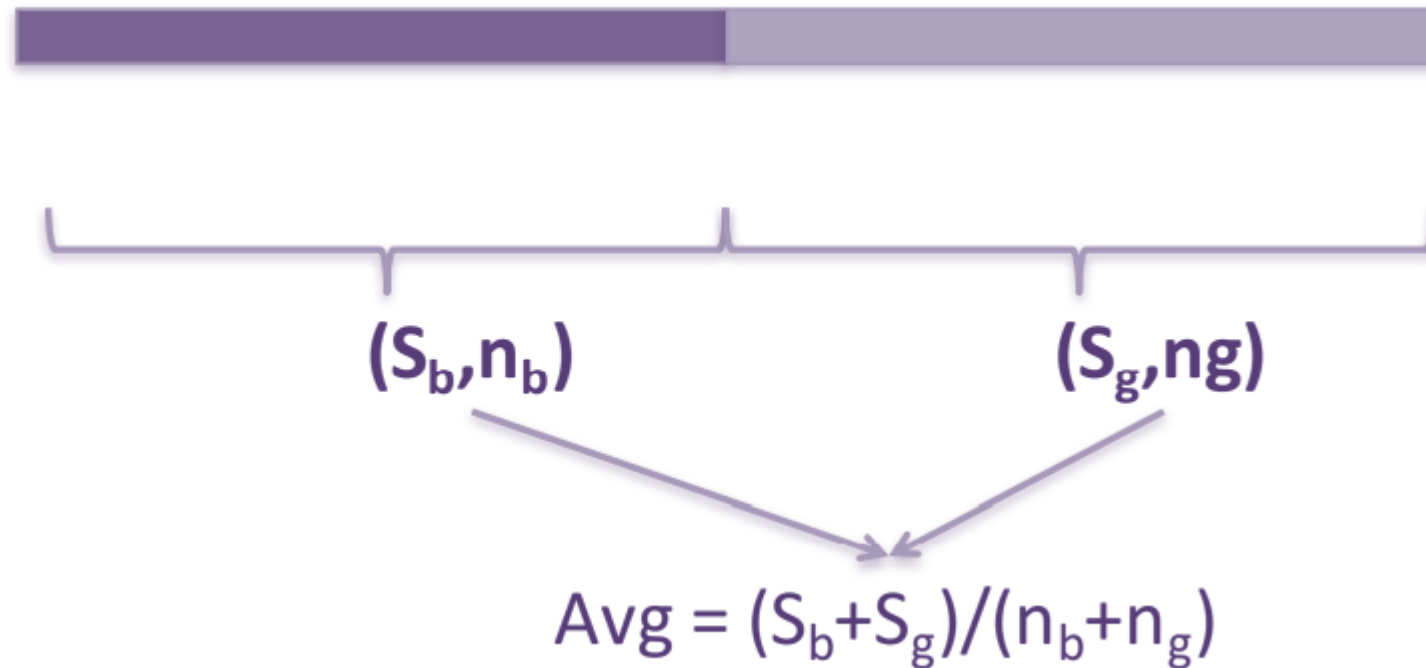
MapReduce – Refinement : Combiners

Average



MapReduce – Refinement : Combiners

Average



MapReduce – Refinement : Combiners

For the combiners refinement, the Average function was rewritten as a function of commutative and associative aggregations

However, not all operations can be rewritten as such functions.

For example, for a Median function (Find the element 'in the middle' of a value of set) it cannot be written as commutative and associative aggregations.

Thus, all values must be shipped to the reducer where computation of the Reduce function will take place.



MapReduce – Refinement: Partitions

2. Partition function

Why ?

- We want to control how keys get partitioned

What ?

- By default, system uses a default partition function:
 - Hash(key) mode R
- This function can be overridden by the user